

The logo for GA Guardian, featuring a stylized 'GA' in a bold, sans-serif font followed by the word 'GUARDIAN' in a smaller, all-caps, sans-serif font.

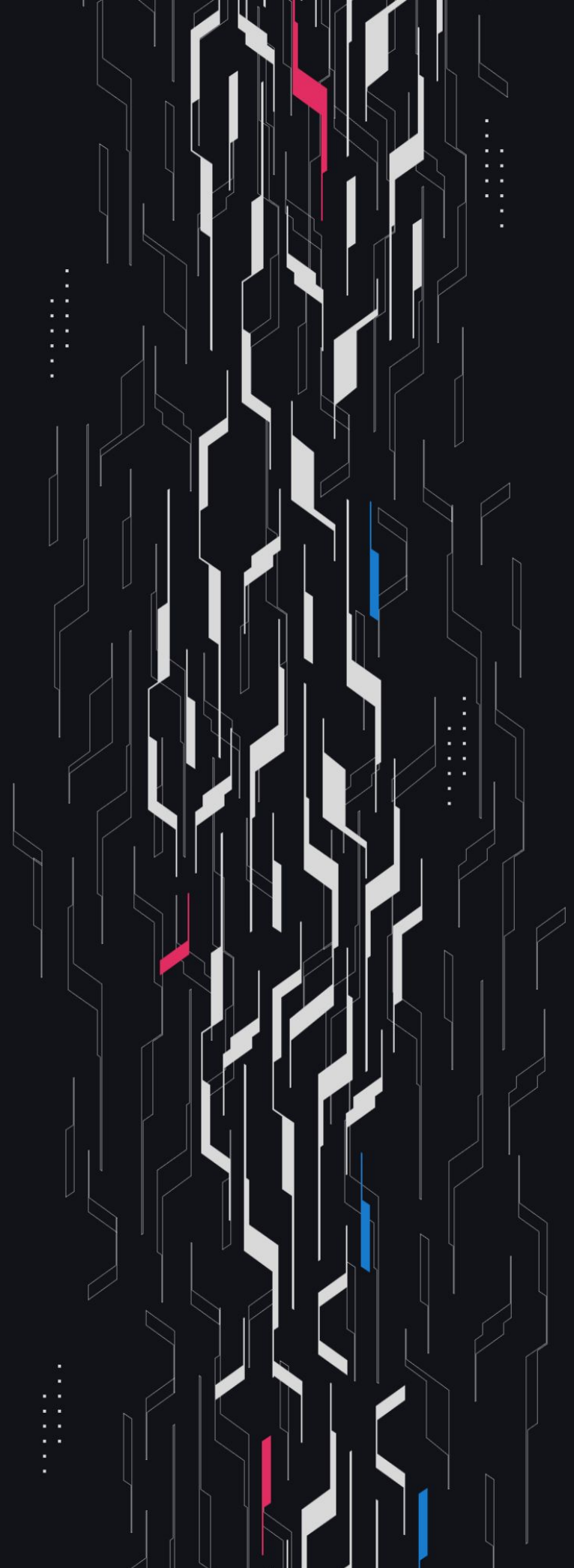
GA GUARDIAN

Jupiter

JUPUSD Updates

Security Assessment

March 16, 2026



Summary

Audit Firm Guardian

Prepared By Zdravko Hristov, Minato Namakazi

Client Firm Jupiter

Final Report Date 03/16/2026

Audit Summary

Jupiter engaged Guardian to review the security of its JupUSD Updates. From the 02/23/2026 to the 02/26/2026, a team of 2 auditors reviewed the source code in scope. All findings have been recorded in the following report.

Confidence Ranking

Given the lack of High and Critical issues detected during the main review, Guardian assigns a Confidence Ranking of 5 to the protocol. Guardian advises the protocol to consider periodic review with future changes. For detailed understanding of the Guardian Confidence Ranking, please see the rubric on the following page.

Note: Fixes to the findings uncovered in the remediation review have not been reviewed by Guardian.

✓ Verify the authenticity of this report on Guardian's GitHub: <https://github.com/guardianaudits>

 Code coverage & PoC test suite:

Guardian Confidence Ranking

Confidence Ranking	Definition and Recommendation	Risk Profile
5: Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4: High Confidence	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. Recommendation: Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3: Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases. Recommendation: Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2: Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. Recommendation: Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week. Or additional High/Critical findings uncovered in remediation review which have not been resolved and confirmed by Guardian.
1: Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥ 5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. Recommendation: Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥ 5 High/Critical findings and overall systemic flaws.

Table of Contents

Project Information

Project Overview 5

Audit Scope & Methodology 6

Smart Contract Risk Assessment

Findings & Resolutions 9

Addendum

Disclaimer 22

About Guardian 23

Project Overview

Project Summary

Project Name	Jupiter
Language	Solidity
Codebase	https://github.com/jup-ag/jup-stablecoin
Commit(s)	Main Review commit: f1102c984358f873fdd39d8e0e80ae191c8b3631 Remediation Review commit: 05d4176a9cb1dc186457af686366e973a3d9bed6

Audit Summary

Delivery Date	03/16/2026
Audit Methodology	Static Analysis, Manual Review, Test Suite, Contract Fuzzing

Vulnerability Summary

Vulnerability Level	Total	Pending	Declined	Acknowledged	Partially Resolved	Resolved
● Critical	0	0	0	0	0	0
● High	0	0	0	0	0	0
● Medium	0	0	0	0	0	0
● Low	0	0	0	0	0	0
● Info	11	2	0	5	0	4

Audit Scope & Methodology

Contract Updates:

<https://github.com/jup-ag/jup-stablecoin/compare/da2cd2b62d81b493f145fd0f81338a6cb5a973ed...d30bc4654d1f60a379b05aa5d29657d7dc9a5f5b>

Audit Scope & Methodology

Vulnerability Classifications

Severity	Impact: <i>High</i>	Impact: <i>Medium</i>	Impact: <i>Low</i>
Likelihood: <i>High</i>	● Critical	● High	● Medium
Likelihood: <i>Medium</i>	● High	● Medium	● Low
Likelihood: <i>Low</i>	● Medium	● Low	● Low

Impact

- High** Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.
- Medium** A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.
- Low** Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

Likelihood

- High** The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.
- Medium** An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.
- Low** Unlikely to ever occur in production.

Audit Scope & Methodology

Methodology

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.
Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.

Findings & Resolutions

ID	Title	Category	Severity	Status
I-01	Admin Can Close Operator Accounts For Rent	Access Control	● Info	Acknowledged
I-02	Lagging Oracle Lowers Redemption Price	Informational	● Info	Pending
I-03	Typo In Slot_treshold Naming	Typo	● Info	Resolved
I-04	Doves Staleness Check Is Stricter	Validation	● Info	Resolved
I-05	Overriding Fee Race Conditions	Warning	● Info	Acknowledged
I-06	Slippage Protection On Redeem May Not Be Exact	Best Practices	● Info	Acknowledged
I-07	Restart Slot Check May Allow Old Data	Warning	● Info	Acknowledged
I-08	Positive Expo Handled As Absolute Scale	Informational	● Info	Resolved
I-09	Single Stalesness_threshold Is Reused	Informational	● Info	Acknowledged
I-10	Confidence Semantics Differ Across Oracles	Math	● Info	Resolved

I-01 | Admin Can Close Operator Accounts For Rent

Category	Severity	Location	Status
Access Control	● Info	operator.rs: 56-73	Acknowledged

Description

delete_operator() only requires the caller's operator to have OperatorRole::Admin and prevents self-deletion, but does not restrict which deleted_operator can be closed. As a result, any admin can close any other Operator account by passing it as deleted_operator, and can select any payer to receive the reclaimed rent via close = payer.

Recommendation

If intended, document this behavior in DeleteOperator/delete_operator() so it's an explicit design choice. Otherwise, constrain deleted_operator to the caller's operator_authority (or a designated governance authority) and/or hardcode the close destination to a treasury account instead of a caller-supplied payer.

Resolution

Jupiter: Acknowledged.

I-02 | Lagging Oracle Lowers Redemption Price

Category	Severity	Location	Status
Informational	● Info	Jup-stable	Pending

Description

when a vault has ≥ 2 oracles, `parse_oracles` always selects the MIN price and redeem uses that MIN for both the `max_oracle_price_usd` check and payout math, so a lagging/low feed (spread allowed up to 200bps) can make redeems pay 1:1 (minus fee only) even when another oracle shows $> \$1$, meaning the higher oracle price would be more profitable to the protocol while redeeming

Recommendation

consider setting fees higher than 10 bps to cover the worst case divergence, If this is intended, we are sending this one as information

If you are planning to support a high volatility stable token as collateral, use the max oracle in redeem and keep using the min oracle in minting

Resolution

Jupiter: Pending.

I-03 | Typo In Slot_treshold Naming

Category	Severity	Location	Status
Typo	● Info	oracle.rs: 53	Resolved

Description

The slot_tresholdvariable in from_switchboard_on_demand() spells threshold incorrectly.

Recommendation

Fix the typo

```
- let slot_treshold = stalesness_threshold * 1000 / clock::DEFAULT_MS_PER_SLOT;  
+ let slot_threshold = stalesness_threshold * 1000 / clock::DEFAULT_MS_PER_SLOT;
```

Resolution

Jupiter: Resolved.

I-04 | Doves Staleness Check Is Stricter

Category	Severity	Location	Status
Validation	● Info	oracle.rs: 97	Resolved

Description

`from_doves()` enforces `price.timestamp + s > clock.unix_timestamp`, which rejects prices exactly at the staleness boundary. The `from_pyth_v2()` path uses `get_price_no_older_than()` (`>=`), and the Switchboard path checks `last_update_timestamp + staleness_threshold >= clock.unix_timestamp`. This creates inconsistent staleness semantics across oracles.

Recommendation

Align the boundary check by using `>=` in `from_doves()` or document the stricter Doves behavior for operators.

Resolution

Jupiter: Resolved.

I-05 | Overriding Fee Race Conditions

Category	Severity	Location	Status
Warning	● Info	benefactor.rs	Acknowledged

Description

Each benefactor stores an array of 4 fee overrides that can be modified or deleted by an operator with the `BenefactorManager` role. To do that, the operator has to provide the index of the fee override they wish to modify.

This creates the possibility of race conditions where the data at that index changed before that and the end state ends up being different than what's expected.

For example, if slot 0 is empty and two different operators decide to fill it - one for USDC and the other for USDT. After their transactions execute, only one of the overrides will remain, depending on which transaction executed first.

Recommendation

Be aware of this behavior and monitor the fee overrides to ensure the state is being updated as intended.

Resolution

Jupiter: Acknowledged.

I-06 | Slippage Protection On Redeem May Not Be Exact

Category	Severity	Location	Status
Best Practices	● Info	user.rs: 299-312	Acknowledged

Description

The `redeem_amount` is compared against `min_amount_out` to ensure the user is protected against unexpected redeem results.

```
require!(
  redeem_amount >= min_amount_out,
  JupStableError::SlippageToleranceExceeded
);
```

Then the code enforces that the amount that leaves `vault_token_account` is the exact `redeem_amount`

```
let amount_before = ctx.accounts.vault_token_account.amount;
transfer_checked(
  ctx.accounts
    .withdraw_collateral()
    .with_signer(&[authority_seeds!(config.authority_bump)]),
  redeem_amount,
  ctx.accounts.vault_mint.decimals,
)?;
ctx.accounts.vault_token_account.reload()?;
let amount_after = ctx.accounts.vault_token_account.amount;
require!(
  amount_after == amount_before - redeem_amount,
  JupStableError::InsufficientAmount
);
```

However, this doesn't guarantee the user received the same amount. For example, if the underlying token has a fee feature that has been turned on after the mint path was used, the user may receive an amount under their specified slippage parameter.

Recommendation

Consider using the delta in the slippage check as well.

Resolution

Jupiter: Acknowledged.

I-07 | Restart Slot Check May Allow Old Data

Category	Severity	Location	Status
Warning	● Info	I-07-3138bda5828c819c9e42eb6987ce5cd5?pvs=21	Acknowledged

Description

The `last_update_slot()` check only requires that one submission happened after `last_restart_slot`. However, `get_value()` still computes the median over all submissions within `slot_treshold` slots, which can include pre-restart submissions if the restart slot falls inside the staleness window. This means the post-restart gate does not fully prevent using pre-restart data.

Recommendation

Document this if it's an accepted behavior.

Resolution

Jupiter: Acknowledged.

I-08 | Positive Expo Handled As Absolute Scale

Category	Severity	Location	Status
Informational	● Info	oracle.rs	Resolved

Description

from_doves() converts the price using `Decimal::from_i128_with_scale(price.price, price.expo.abs())`. This implicitly assumes expo is non-positive. If expo is ever positive, using `abs()` would divide by 10^{expo} instead of multiplying, producing an incorrect price.

Recommendation

Pyth usually uses negative expos, so keep this in mind if somehow one of the configured oracles may return positive one.

Resolution

Jupiter: Resolved.

I-09 | Single Stalesness_threshold Is Reused

Category	Severity	Location	Status
Informational	● Info	oracles.rs	Acknowledged

Description

parse_oracles() forwards the same stalesness_threshold to from_pyth_v2(), from_switchboard_on_demand(), and from_doves(). If any of the oracles update times differs, stale price may be used.

Recommendation

Ensure the staleness used will be suitable for all of the configured oracles.

Resolution

Jupiter: Acknowledged.

I-10 | Confidence Semantics Differ Across Oracles

Category	Severity	Location	Status
Math	● Info	oracles.rs	Resolved

Description

from_pyth_v2() compares Pyth price.conf (an explicit confidence interval half-width) against MAX_CONFIDENCE_BPS, while from_switchboard_on_demand() compares std_dev() (standard deviation of submissions) against the same threshold. and because of that the end result is not exactly the same. The std must be in MAX_CONFIDENCE_BPS of the actual price.

Recommendation

If the current behavior is intended, document the assumption in the oracle handling. Otherwise implement different check for the std.

Resolution

Jupiter: Resolved.

Remediation Findings & Resolutions

ID	Title	Category	Severity	Status
I-01	Manipulating Oracle Can Affect Redeems	Informational	● Info	Pending

I-01 | Manipulating Oracle Can Affect Redeems

Category	Severity	Location	Status
Informational	● Info	oracle.rs	Pending

Description

The final decision for pricing redeems is to use the minimum price returned by all the configured oracles in order to minimize spread. This increases the potential impact of oracle manipulation, as now only one of the oracles has to return a lower price in order to affect redeems.

For example, if the real price is 1.01, a manipulated oracle may return 0.999 and trigger redeem with the 1 branch. It's expected that the fee charged will compensate for this, but it still depends on the exact configuration.

Recommendation

Be aware of this and keep monitoring the oracles after deployment.

Resolution

Jupiter: Pending.

Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit <https://guardianaudits.com>

To view our audit portfolio, visit <https://github.com/guardianaudits>

To book an audit, message <https://t.me/guardianaudits>